

OPENER: Open Partitioning Embedding for Named Entity Recognition

Thibault Garel^{1*}, Faiza Belbachir¹ and Assia Soukane¹

^{1*}LyRIDS, ECE Engineering School, Paris, 75015, France.

*Corresponding author(s). E-mail(s): garel.thibault14@gmail.com;
Contributing authors: fbelbachir@ece.fr; assia.soukane@ece.fr;

Abstract

Named Entity Recognition (NER), a foundational Natural Language Processing task, extracts entities (persons, dates, locations) from raw text. Yet, real-world applications rarely afford a static class inventory. Open-world NER therefore assigns types not fixed at training time, by training a dedicated encoder or prompting billion-parameter language models, compared on accuracy alone, ignoring energy cost. We present OPENER, an open-world NER pipeline redefining the optimal trade-off between accuracy, latency, and energy frugality. Uncommonly, this transfer-learning pipeline trains no encoder from scratch: a frozen mention detector, a Matryoshka-truncatable embedder sharpened by contrastive fine-tuning with error-driven hard-negative mining, and a swappable typing head. OPENER-ZS, the zero-shot head, types mentions against label-name prototypes without target labels, while OPENER-Sup fits a lightweight supervised probe on labelled classes, trading openness for accuracy. We benchmark OPENER against GLiNER, GNER, OWNER and a 4-bit Qwen2.5-1.5B model on 13 datasets along three axes (Adjusted Mutual Information (AMI), latency, energy and CO₂), separating gold-mention typing from end-to-end, exposing order-of-magnitude gaps that accuracy-only evaluations hide. Over three retraining seeds, both variants lead their category: OPENER-ZS is the most accurate zero-shot system end-to-end (39.5 AMI vs 38.9 for GLiNER-L, 34.3 for OWNER) while staying frugal (1.7 Wh, 180 ms, versus 14.3 Wh, 616 ms for OWNER). With target labels, OPENER-Sup is the most accurate overall (40.1 AMI). On gold mentions, typing alone reaches 62.3 AMI versus OWNER’s 43.0, pinning the open-world bottleneck on mention detection, the clearest axis for future gains. We release code, configurations, fine-tuned embedders, and benchmark.

Keywords: Named entity recognition, Open-world NER, Transfer learning, Zero-shot learning, Contrastive learning, Energy efficiency

1 Introduction

Named Entity Recognition (NER) is a core task of Natural Language Processing (NLP). It locates and classifies key elements of unstructured text into predefined categories (e.g., extracting "Marie Curie" as a PERSON or "1898" as a DATE) and supports the analysis of legal or medical documents, the routing of support tickets and search. However, real-world NER rarely operates with a static label set, as novel categories emerge across new domains. Traditional models assign contiguous sequences of tokens, called spans, to fixed classes, and this rigid training leaves them blind to unobserved types, such as "radium" as a CHEMICAL [27]. To avoid re-annotating data and retraining from scratch, *open-world* NER recognises and types entities without relying on a frozen training inventory.

Notable examples of open-world NER models include: GLiNER [48] scores candidate spans against the target type names supplied as queries in a single forward pass, while GNER [8] recasts NER as text generation with a sequence-to-sequence model. Both type entities whose categories were not fixed at training time, at the cost of training a dedicated model. Another line prompts or instruction-tunes billion-parameter language models, such as UniversalNER [50] and GoLLIE [34], which are accurate but slow and often too large for commodity hardware. Even a small quantized model that fits, such as Qwen2.5-1.5B-Instruct [31] in 4-bit, remains comparatively slow and costly. OWNER [10] trains a dedicated entity encoder with a triplet objective and clusters the mentions into dynamically named types, but it still trains an encoder from scratch and pays a heavy per-sentence inference cost. Across all of these, evaluation is reported almost exclusively on accuracy, although compute, latency and energy decide what can actually be deployed.

We present **OPENER** (Open **P**artitioning **E**mbedding for **N**amed **E**ntity **R**ecognition), an open-world NER pipeline designed for accuracy, latency and energy at once and assembled entirely from pretrained components, so that no encoder is trained from scratch. A frozen mention detector proposes spans, a pretrained Matryoshka sentence embedder, sharpened by a light contrastive fine-tuning and an error-driven hard-negative mining pass, maps each span to a vector, and a typing head assigns a type. The three stages are independently swappable, which lets us select each component on measured evidence (subsection 4.7). The typing head has two operating points: OPENER-ZS, a label-free zero-shot head that we put forward as the open-world default, and OPENER-Sup, a supervised probe for domains whose classes are fixed and labelled.

Our contributions are as follows.

- **A modular open-world NER pipeline built by transfer learning.** To the best of our knowledge, OPENER is the first open-world NER pipeline assembled entirely from ready-to-use pretrained components. Only the embedder is lightly fine-tuned, first contrastively and then by error-driven hard-negative mining on the most confused type pairs, which improves typing-on-gold AMI by 8.5 points over contrastive fine-tuning alone.
- **OPENER-ZS**, a typing head that needs no target labels. It matches each mention to prototypes built from the label names, refines them transductively on the

unlabelled test mentions and fuses the result with the detector’s own zero-shot type guess. It is the most accurate zero-shot system end-to-end on our benchmark, ahead of GLiNER-L, GNER and OWNER under an identical protocol.

- **OPENER-Sup**, a balanced linear probe (LinearSVC with class re-weighting) for fixed and labelled label sets. It trades openness for accuracy and is the most accurate configuration of the benchmark, both end-to-end and on gold mentions.
- **A latency and energy-aware open-world NER benchmark** on 13 datasets that reports quality, latency and energy jointly and separates typing-on-gold from end-to-end. It shows that mention detection, not typing, is the dominant bottleneck, and that a small 4-bit instruction-tuned LLM (Qwen2.5-1.5B-Instruct) is both the least accurate and, per sentence, the most expensive system (on a 150-sentence subsample). All OPENER results are averaged over three embedder-retraining seeds.

The rest of the paper is organised as follows. section 2 reviews related work, section 3 details the OPENER pipeline, section 4 presents the benchmark and ablations, and section 5 concludes.

2 Related Work

We position OPENER with respect to closed-set, open-world and LLM-based NER, to contrastive entity representations, and to energy-aware machine learning.

2.1 Closed-set and Span-based NER

Traditional NER is cast as supervised sequence labeling over a fixed inventory of types. Early neural taggers combined recurrent encoders with a CRF layer [18], and fine-tuning a pretrained transformer such as BERT [7] further increased in-domain accuracy. Span-based formulations classify candidate spans directly, which eases nested entities [27], either with a biaffine scorer [47] or by recasting typing as machine reading comprehension over natural-language type descriptions [21], a use of label semantics our zero-shot head also exploits. These models reach high accuracy on in-domain benchmarks such as CoNLL-2003 [39], as surveyed in [19], but their label set is frozen at training time, and adding a type requires re-annotating data and retraining.

2.2 Open-world and Zero-shot NER

Open-world NER recognises entity types that were not fixed at training time, and zero-shot NER, its most demanding setting, types entities without any labelled example of the target types. GLiNER [48] treats the candidate type names as queries to a DeBERTaV3 encoder [11] and scores every span against them in a single forward pass. GNER [8] casts NER as generation with a sequence-to-sequence model [32] that regenerates the sentence with inline type tags and explicitly models non-entity tokens. Closest to our work, OWNER [10] decouples mention detection from typing: it embeds detected mentions with a contrastively refined encoder and clusters them into dynamically named types. Zero-shot typing can also rely on natural-language

type descriptions [1], a premise our label-name prototypes share, or on a general-purpose LLM that extracts entities directly, as in ChatIE [44]. OPENER adopts the decoupled detect-then-type view of OWNER, but replaces the trained encoder and the unsupervised clustering with a pretrained Matryoshka embedding space and a lightweight typing head.

2.3 LLM-based and Few-shot NER

LLM-based NER prompts or instruction-tunes large language models to extract entities from a description of the target types, while few-shot NER learns new types from a handful of labelled examples. UniversalNER [50] distills a chat model into a LLaMA-based student [40] over tens of thousands of entity types, and GoLLIE [34] conditions a code language model on annotation guidelines. Few-shot methods reduce supervision further, for instance with token-level contrastive objectives such as CONTaiNER [3]. These methods are accurate but rest on billion-parameter models whose latency and energy use are substantial and which may not fit commodity hardware. As a runnable reference point at the small end of this family, we include Qwen2.5-1.5B-Instruct [31] in 4-bit, prompted to return entities as JSON (subsection 4.5).

2.4 Entity Representations and Contrastive Learning

Entity representations map each mention to a vector whose geometry reflects its type, and contrastive learning shapes this geometry by pulling same-type mentions together and pushing different types apart. Nomic Embed [28], from the Sentence-BERT line of work [33], is trained with the Matryoshka objective [16], so its representations can be truncated without retraining. We sharpen its entity-type geometry with a triplet objective in the spirit of FaceNet [35], and mining hard negatives, as in GCSE [17], is close in spirit to our error-driven hard-negative mining. The base encoder is chosen empirically among strong open encoders (E5 [43], BGE [46], MPNet [37] and mxbai [20]), among which Nomic v1.5 performs best (subsection 4.7). Embedding labels and entities in a shared space [42] is a recurring idea in low-resource NER, and entity types can also be modeled with Gaussian mixtures fitted by expectation-maximization [4]. Unlike methods that train a dedicated encoder, OPENER keeps a general-purpose space and only applies a light contrastive fine-tuning.

2.5 Efficient and Green AI

Prior work has quantified the cost of training NLP models [38], promoted efficiency as an evaluation criterion under the *Green AI* agenda [36], and called for the systematic reporting of energy and carbon [12], which tools such as CodeCarbon [26] make practical. At inference time, general-purpose generative models are orders of magnitude more expensive per query than task-specific ones [25]. Yet open-world NER systems are still compared almost exclusively on quality, and existing methods either train a dedicated encoder [10] or query costly LLMs [34, 50]. OPENER reuses pretrained components with only a light contrastive step and is evaluated on three axes: quality, measured by Adjusted Mutual Information [41], inference speed, and energy.

3 Method

3.1 Problem Formulation

Given a sentence, an open-world NER system must return a set of typed mentions whose type inventory is *not* fixed at training time. Formally, from a sentence s , a mention detector produces the set of candidate mentions $\mathcal{M}(s) = \{m_i\}$. Each mention m_i is then passed, together with its sentence s , to an embedder f_θ that maps it *in context* to a D -dimensional representation vector e_i . Finally, a typing head assigns m_i a predicted type $\hat{y}_i$ drawn from the type inventory $\mathcal{T}$,

$$e_i = f_\theta(s, m_i) \in \mathbb{R}^D, \quad \hat{y}_i \in \mathcal{T}, \quad (1)$$

where the type inventory $\mathcal{T}$ is unknown at training time.

We adopt the same evaluation protocol as OWNER [10]. To keep the end-to-end score honest, detection errors must be penalised rather than quietly ignored. We align predicted and gold mentions by their exact character offsets, so a prediction is a match only when it covers exactly the same span as a gold mention. Any mention left unmatched is then a detection error of one of two kinds. A gold mention that no prediction matches is a *false negative* (a missed entity, for example, the detector finds “Marie Curie” but misses “radium” in “Marie Curie discovered radium”). A predicted mention that matches no gold entity is a *false positive* (a spurious span, for example, the detector wrongly proposes “discovered” as an entity). Each unmatched mention receives its own distinct sentinel type, so that these errors count as mismatches when computing the AMI instead of being ignored.

Type-assignment quality is then measured by Adjusted Mutual Information (AMI) [41] between the gold type partition U and the predicted partition V over the aligned mentions,

$$\text{AMI}(U, V) = \frac{\text{MI}(U, V) - \mathbb{E}[\text{MI}(U, V)]}{\frac{1}{2}[H(U) + H(V)] - \mathbb{E}[\text{MI}(U, V)]}, \quad (2)$$

where MI is the mutual information and H the entropy, and the denominator uses the arithmetic-mean normalisation (the scikit-learn default). Intuitively, mutual information measures how much knowing a mention’s predicted type reduces the uncertainty about its true (gold) type, and it is zero exactly when the two partitions are statistically independent (the prediction tells us nothing about the truth). The *adjusted* form further subtracts the agreement expected by chance, so a random typing scores close to 0 whatever the number of types, while a perfect one scores 1. A key advantage in the open-world setting is that AMI does not require the two label vocabularies to coincide, since the predicted type names need not match the gold ones.

We report two protocols that are never mixed in the same comparison: *typing-on-gold*, where the typing component is applied to gold spans and detection is bypassed, and *end-to-end*, where mentions come from the detector and the sentinels above are active.

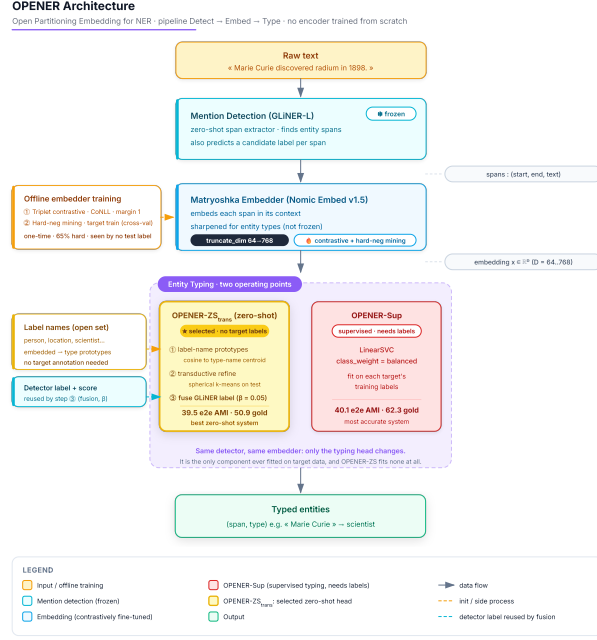


Fig. 1 The OPENER pipeline (Detect → Embed → Type): a frozen GLiNER detector proposes spans, a Matryoshka embedder (Nomic, contrastively fine-tuned) maps each span in context to a vector, and a typing head assigns a type. The typing head (the only target-fitted component) has two operating points: the *zero-shot* OPENER-ZS_{trans}(★, selected) and OPENER-Sup (*supervised*, the upper bound). All upstream components are ready-to-use

3.2 Architecture Overview

OPENER is a three-stage pipeline, Detect $\rightarrow$ Embed $\rightarrow$ Type (Figure 1), assembled from ready-to-use components so that no encoder is trained from scratch. Three principles guide the design.

- A pretrained detector and a pretrained sentence embedder are reused as-is.
- A light contrastive fine-tuning, refined by error-driven hard-negative mining, turns the general-purpose embedding into one that is discriminative for entity *types*.
- The embedder is Matryoshka-truncatable, exposing a compute/quality knob.

Only the typing head is fitted on target-side material, and it comes in two forms: a zero-shot form, OPENER-ZS, that uses no target labels at typing time and OPENER-Sup, a supervised linear probe that is the strongest configuration but requires target labels.

3.3 Mention Detection

Mention detection is delegated to GLiNER [48], a frozen, ready-to-use detector that scores every span against the candidate type names in a single forward pass. Decoupling detection from typing lets each component be chosen and measured independently, and it lets the typing head operate on any span proposer. We use GLiNER-L by default, as it yields the best AMI. The small and medium variants trace a size/efficiency trade-off (see subsection 4.7 for the full detector comparison). As shown in subsection 4.5, the detector is the shared end-to-end bottleneck: its recall on specialised domains, not the typing head, caps end-to-end quality.

3.4 Entity Embedding

Detected spans are embedded with Nomic Embed v1.5 [28], a sentence embedder trained with the Matryoshka objective [16], whose representation can be truncated to a lower dimension and renormalised without retraining. Each mention is embedded *in context* rather than in isolation: the span is presented together with its surrounding sentence, so that polysemous surface forms (e.g. Apple as a company versus a fruit) are disambiguated. This yields a vector $e_i \in \mathbb{R}^D$ with D truncatable in $\{768, 512, 256, 128, 64\}$. We keep the full $D=768$, which gives the best AMI (see subsection 4.7 for the full dimension sweep). Truncation keeps only the first d coordinates of the embedding ($d \leq D$) and rescales them back to unit length,

$$e_i^{(d)} = e_{i,1:d} / \|e_{i,1:d}\|_2, \quad (3)$$

where $e_i \in \mathbb{R}^D$ is the in-context embedding of mention m_i , $e_{i,1:d}$ is its truncation to the first d components (the coordinates $d+1$ to D are simply dropped), and $\|e_{i,1:d}\|_2 = \sqrt{\sum_{k=1}^d e_{i,k}^2}$ is its Euclidean (L_2) norm, that is the geometric length of the vector. Dividing by this norm rescales $e_i^{(d)}$ to length 1, so that embeddings truncated at different d remain comparable through cosine similarity. This shrinks the

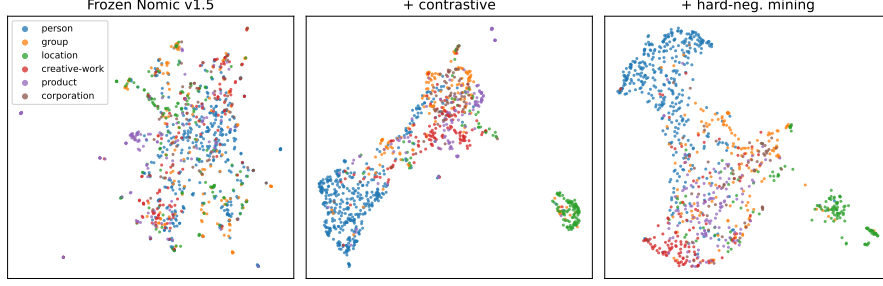


Fig. 2 UMAP of WNUT-17 (held-out) entity embeddings, coloured by gold type, across the three embedder stages: frozen Nomic v1.5, + contrastive fine-tuning, + hard-negative mining. The contrastive stages progressively separate the types into compact clusters on a domain unseen during training

stored embedding and the typing-head fit cost but leaves the forward pass unchanged (subsection 4.7).

Still, the frozen embedding is only weakly discriminative for entity types (25.8 AMI, subsection 4.7), so we sharpen its geometry with *contrastive fine-tuning*, in two stages, while keeping the backbone otherwise intact.

3.4.1 Stage 1: contrastive

We fine-tune the embedder with a Triplet Margin Loss $\mathcal{L}$ [35] (using sentence-transformers [33]), with margin $\alpha = 1$, on the CoNLL-2003 [39] training spans. We use CoNLL-2003 because its generic, domain-neutral label set makes it the standard source domain of our benchmark (Table 1). Since the embedder is only lightly fine-tuned on it, we exclude CoNLL-2003 from the reported average, so as not to bias the results in our favour, following the same protocol as OWNER. A triplet (a, p, n) takes an anchor span a , a positive p of the *same* type, and a negative n of a *different* type, with at most two triplets per anchor. Writing α for the margin, each triplet contributes

$$\begin{aligned} \mathcal{L}(a, p, n) &= \max(0, \|e_a - e_p\|_2 - \|e_a - e_n\|_2 + \alpha), \\ \alpha &= 1, \end{aligned} \tag{4}$$

which pulls same-type spans together and pushes different-type spans apart, i.e. it decreases the embedding distance $\|e_a - e_p\|_2$ while increasing $\|e_a - e_n\|_2$. Training runs for 3 epochs with batch size 16, learning rate 2×10^{-5} and 100 warmup steps. This single step lifts typing-on-gold AMI from 25.8 to 54.0. The UMAP projection in Figure 2 shows how Stage 1 reshapes the embedding space.

3.4.2 Stage 2: error-driven hard-negative mining

We then mine the model’s own confusions. We defer this to Stage 2, after the Stage-1 warm-up, to avoid overfitting to these hard cases. On each target training split, we run a cross-validated typing pass (so no test label is ever seen) and collect the type pairs that are most often confused, such as *writer/person*, *album/band* and *country/location*. For each such error, we form a *hard* triplet whose anchor is the mistyped

span (true type L), whose positive is another span of L , and whose negative is a span of the confused type C . Hard triplets are mixed with correctly-typed (easy) triplets for diversity, targeting a 65% hard fraction, and the Stage-1 embedder is fine-tuned further with the same optimiser settings. This ratio must be controlled carefully to preserve generalisation. The selected run mines 8000 triplets over 3 epochs and reaches 62.5 AMI at the released seed (+8.5 over Stage 1, improving on 12 of the 13 sets). Retraining the full two-stage recipe under three seeds yields 62.3 ± 1.0 (subsection 4.3). This embedder is shared unchanged by every typing head below. The UMAP projection in Figure 2 also reveals the additional effect of Stage 2 on the embedding space.

3.5 Entity Typing

The embedding above feeds a typing head, the only component fitted on target-side material. It has two operating points, presented in turn: a label-free *zero-shot* head, OPENER-ZS, which we adopt as the open-world default, and a *supervised* probe, OPENER-Sup, a stronger but label-dependent upper bound.

3.5.1 Zero-shot typing (OPENER-ZS)

To remove target supervision at typing time, each type t is represented by a prototype p_t : the ℓ_2 -normalised centroid of the embeddings of its name (its anchor words) in the contrastive space,

$$p_t = \bar{e}_t / \|\bar{e}_t\|_2, \quad \bar{e}_t = \frac{1}{|\text{name}(t)|} \sum_{w \in \text{name}(t)} f_\theta(w), \quad (5)$$

where t is a candidate type, $\text{name}(t)$ is the set of words forming its name (its anchor words, e.g. “person” or “astronomical object”), $f_\theta(w)$ is the embedding of anchor word w in the contrastive space, and $\bar{e}_t$ is their average (the centroid), which dividing by its Euclidean norm $\|\bar{e}_t\|_2$ turns into a unit-length prototype p_t (the averaging factor cancels under this normalisation). A mention m_i is then assigned the type of the nearest prototype by cosine similarity, $\hat{y}_i = \arg \max_t \cos(e_i, p_t)$. This inductive nearest-centroid head is OPENER-ZS_{ind}. Two label-free refinements improve this head without using any target label. First, a *transductive* refinement runs one pass of spherical k -means over the unlabelled test mentions, seeded by the prototypes. Second, a *detector fusion* reuses a signal the prototype head otherwise discards: GLiNER already predicts a zero-shot type for each span with confidence g_i , which we add to the similarity score before taking the arg-max,

$$s_t(i) = \cos(e_i, p_t) + \beta g_i \mathbb{I}[t = \ell_{\text{GLiNER}}(i)], \quad (6)$$

where $s_t(i)$ is the fused score of assigning type t to mention m_i , $\cos(e_i, p_t)$ is the prototype similarity above, $g_i \in [0, 1]$ is GLiNER’s own confidence for the type $\ell_{\text{GLiNER}}(i)$ it predicted, and $\mathbb{I}[\cdot]$ is the indicator function, equal to 1 when t is exactly that predicted type and 0 otherwise. The fusion weight $\beta=0.05$ was found by a small empirical sweep and is deliberately kept low, so the detector only nudges the arg-max, and the

assigned type is the highest-scoring one, $\hat{y}_i = \arg \max_t s_t(i)$. The refined, fused head is OPENER-ZS_{trans}, the selected zero-shot operating point: it reaches 39.5 AMI end-to-end (mean over three retraining seeds), ahead of every compared zero-shot system, while using no target labels (subsection 4.5).

3.5.2 Supervised probe (OPENER-Sup)

When a domain’s entity classes are fixed and labelled, the typing head can instead be a balanced linear classifier (LinearSVC with the LIBLINEAR solver [9]) fitted one-vs-rest on the target’s training labels. For each type c , it minimises a squared-hinge objective with balanced class weights,

$$\min_{w_c, b_c} \frac{1}{2} \|w_c\|_2^2 + C \sum_i \omega_{y_i} \max(0, 1 - z_{ic}(w_c^\top e_i + b_c))^2, \quad (7)$$

where, for each type c , w_c and b_c are the weight vector and bias of its one-vs-rest hyperplane, e_i is the (frozen) embedding of training mention i and y_i its gold type, $z_{ic}=+1$ if $y_i=c$ and -1 otherwise, and C is the regularisation strength that trades margin width against training errors. The squared-hinge term $\max(0, 1 - z_{ic}(w_c^\top e_i + b_c))^2$ penalises mentions on the wrong side of the margin. The balancing weight $\omega_c = n/(|\mathcal{T}|n_c)$, with n the total number of training mentions, $|\mathcal{T}|$ the number of target types and n_c the count of type c , up-weights rare types to counter the benchmark’s heavy imbalance. A mention is finally typed as $\hat{y}_i = \arg \max_c (w_c^\top e_i + b_c)$. This is the strongest configuration but requires target-side labels.

3.6 Evaluation Protocol

We evaluate on the 13 datasets of subsection 4.1, each test split capped at 1000 sentences. Quality is AMI [41] (with macro- F_1 and accuracy), latency is the median (p50) per-sentence time over GPU-synchronised, warmed-up passes, and energy is measured with CodeCarbon [26] under the French grid carbon intensity. The full protocol and instrumentation are detailed in subsection 4.3 and subsection 4.4. The code and the per-experiment configurations are available on GitHub at https://github.com/Thibault-GAREL/LyRIDS_OPENER, and the fine-tuned embedders of OPENER-ZS and OPENER-Sup are available on Hugging Face at <https://huggingface.co/Thibault-GAREL/opener-zs> and <https://huggingface.co/Thibault-GAREL/opener-sup>, respectively.

3.7 Use of Generative AI Tools

During the preparation of this work the first author used Claude Code (Anthropic) in order to scaffold and debug the experiment scripts, to draft and copy-edit the manuscript, and to assist with exploratory data analysis. The research questions, the experimental design, the choice of baselines and datasets, the execution of all experiments, and the interpretation of the results are the first author’s own. After

Table 1 The 13 evaluation sets and their gold entity types.

Dataset	Abbr.	Domain	Test	Entity types (share %)
CrossNER-AI	AI	Encyclopaedic	430	task 12, field 11, product 11, metrics 11, misc 10, algorithm 10, researcher 9, organisation 8, conference 5, person 4, programlang 3, country 2, location 2, university 2
CrossNER-Literature	Lit	Encyclopaedic	416	writer 25, book 18, misc 11, literarygenre 9, person 8, award 6, poem 5, organisation 5, country 4, location 4, magazine 3, event 2
CrossNER-Music	Mus	Encyclopaedic	465	musicalartist 15, musicgenre 15, band 14, location 10, album 10, award 8, song 7, organisation 6, misc 5, country 5, person 2, event 2, musicalinstrument 1
CrossNER-Politics	Pol	Encyclopaedic	651	politicalparty 23, location 14, organisation 12, politician 12, election 10, country 10, person 8, misc 6, event 5
CrossNER-Science	Sci	Encyclopaedic	543	misc 17, scientist 15, astronomicalobject 11, organisation 9, person 7, academicjournal 6, location 6, chemicalcompound 5, protein 5, award 5, university 4, chemicalelement 3, enzyme 3, discipline 2, event 1, country 1, theory 1
WNUT-17	WNUT	Social media	689	person 40, group 15, location 14, creative-work 13, product 12, corporation 6
MIT-Restaurant	Rest	Restaurant search	1000	Location 21, Dish 18, Amenity 16, Hours 15, Restaurant_Name 11, Price 10, Cuisine 5, Rating 4
MIT-Movie	Movie	Movie trivia	1000	TITLE 19, ACTOR 16, YEAR 16, GENRE 16, RATING 9, CHARACTER 9, TRAILER 5, REVIEW 3, RATINGS_AVERAGE 3, PLOT 2, DIRECTOR 2, SONG 1
FabNER	Fab	Manufacturing	1000	MATE 27, CONPRI 21, MANP 15, PRO 11, APPL 6, FEAT 6, PARA 4, MACEQ 3, BIOP 2, CHAR 2, ENAT 2, MANS <1
BioNLP/JNLPBA-2004	Bio	Biomedical	1000	protein 53, cell_type 22, DNA 13, cell_line 10, RNA 1
CoNLL-2003	CoNLL	News	1000	PER 32, ORG 29, LOC 28, MISC 11
GUM	GUM	Multi-genre	1000	abstract 38, person 17, event 12, place 10, object 8, organization 8, time 5, substance 1, animal 1, plant <1
GENTLE	GEN	Multi-genre (challenge)	1000	abstract 55, person 10, event 9, time 8, object 7, place 5, organization 4, animal 1, substance 1, plant <1

"Test" column is the number of test sentences used (capped at 1000). Each type is followed by its share (%) of the test mentions, listed by decreasing frequency (types are the dataset's original labels).

using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

4 Experiments

4.1 Datasets

We evaluate on 13 evaluation sets that span news, social media, spoken queries, encyclopaedic text, biomedical abstracts and manufacturing literature, following the dataset selection of OWNER [10]. CrossNER [24] contributes five domain-specific sets, evaluated separately (artificial intelligence, literature, music, politics, science). The remaining eight are CoNLL-2003 [39], WNUT-17 [5], the MIT-Restaurant and MIT-Movie spoken-query corpora [22], FabNER [15], the BioNLP/JNLPBA-2004 biomedical task [14], and the GUM [49] and GENTLE [2] multi-genre corpora. Label granularity ranges from four coarse types in CoNLL-2003 to seventeen fine-grained categories in CrossNER-Science, including the opaque domain acronyms of FabNER. Two datasets used by OWNER [10], GENIA [13] and the 2014 i2b2/UTHealth clinical corpus, are license-gated and therefore not re-measured here. BioNLP/JNLPBA-2004 serves as our biomedical proxy. To homogenise evaluation cost across systems, we cap each test split at 1000 sentences (the full split is used when it is smaller).

4.2 Baselines

Table 2 Backbones and size of the re-measured baselines and OPENER, in the spirit of OWNER [10].

Model	Backbone	Type	Version	Params	vs OPENER
GLiNER-S	DeBERTaV3	Encoder	deberta-v3-small	50M	0.1×
GLiNER-M	DeBERTaV3	Encoder	deberta-v3-base	200M	0.4×
GLiNER-L	DeBERTaV3	Encoder	deberta-v3-large	330M	0.7×
GNER	T5	Enc.-Dec.	t5-base	220M	0.5×
OWNER ^a	DeBERTaV3+BERT	Encoder	deberta-v3 + bert-base	294M	0.6×
Qwen2.5-1.5B	Qwen2.5	Decoder	qwen2.5-1.5b-instruct	1.5B	3.2×
OPENER (ours)	DeBERTaV3+Nomic	Encoder	deberta-v3-large + nomic-v1.5	467M	1.0×

^a“Type” column is the transformer family. “vs OPENER” column is the total parameter count relative to OPENER’s end-to-end pipeline (467M = GLiNER-L 330M detector + Nomic 137M embedder).

^a OWNER reports 110M for its BERT typing encoder. We count detection + typing for parity with OPENER.

We compare OPENER with representative open-world systems, all run end-to-end on the same splits and scored with the same protocol (subsection 4.3), and summarise their backbones and sizes in Table 2. **GLiNER** [48] is a fast span classifier on a DeBERTaV3 encoder [11], run at three sizes (S/M/L) to trace a size/efficiency trade-off. **GNER** [8] is a generative T5 tagger [32] (base checkpoint), accurate but slowed down by autoregressive decoding. **OWNER** [10], our closest baseline, pairs a DeBERTaV3 detector with a contrastively refined BERT typing encoder [7] and groups the mentions with k -means. It is fully label-free but re-encodes and re-clusters every mention at inference, and we run it in its original environment. **Qwen2.5-1.5B-Instruct** [31], the only generative LLM that fits our 6 GB GPU, runs in 4-bit NF4 [6] and is prompted for JSON-formatted entities. Larger LLM systems such as UniversalNER [50], GoLLIE [34] and ChatIE [44] rely on 7B-parameter backbones [40] that exceed our hardware even in 4-bit. We discuss them qualitatively and refer to the OWNER benchmark [10] for their AMI under a comparable protocol.

4.3 Metrics and Evaluation Protocol

Quality is measured by AMI [41] over the aligned mentions, following OWNER [10] and complemented by macro-averaged F_1 and accuracy. As defined in subsection 3.1, predicted and gold mentions are aligned by exact character offsets, unmatched mentions receive distinct sentinel labels, and the typing-on-gold and end-to-end protocols are never mixed. Efficiency is reported as the median (p50) latency per sentence and as the energy (Wh) and carbon (gCO₂eq) per run. OPENER’s embedder training is stochastic, so we retrain the complete embedder (Stage 1, then Stage 2) under three seeds (7, 42, 123) and report every OPENER cell as mean $\pm$ std. Seed 42 is the released model. The baselines are frozen checkpoints with deterministic inference and are reported as single runs, as is OWNER, trained once with its published protocol. A test-set bootstrap on the released model, a source of uncertainty orthogonal to the retraining seed, is reported in Online Resource 1 (Table S1).

4.4 Experimental Settings

All experiments run on a single consumer-grade GPU, an NVIDIA GeForce GTX 1660 Ti Max-Q with 6 GB of VRAM (CUDA 12.1, PyTorch 2.5.1 [29]). The goal of

Table 3 OPENER configuration.

Component	Setting
Mention detector	GLiNER-L (frozen) [48]
Entity embedder	Nomic v1.5, $D=768$, span-in-context [28]
Contrastive loss	Triplet margin 1 [35]
source	CoNLL-2003 train spans [39]
optimiser	3 epochs, batch 16, lr 2×10^{-5} , 100 warmup
Hard-neg. mining	8000 triplets, 65% hard, 3 epochs
source	target train splits, cross-validated
Supervised typing	LinearSVC, balanced weights [9]
Zero-shot typing	label-name prototype + transductive + fusion ($\beta=0.05$)
Random seeds	7 / 42 / 123 (full embedder retraining per seed, 42 = released)

The detector and embedder are ready-to-use. The contrastive and hard-mining stages produce the shared embedder. The typing head is fitted (supervised) or label-free (zero-shot).

using such modest hardware was to verify that the models could also run on standard personal computers, demonstrating that our approach is highly versatile. We use the HuggingFace Transformers library [45] for baselines, sentence-transformers [33] for the Matryoshka embedder [16, 28], and scikit-learn [30] for the linear classifier, whose solver is LIBLINEAR [9]. Latency is measured with two warmup passes followed by timed passes with GPU synchronisation, reporting p50, p95 and p99 per sentence. Energy is measured with CodeCarbon [26] in offline mode using the French grid carbon intensity. When hardware power counters are unavailable, we fall back to a thermal-design-power estimate and flag the measurement method in the report. All systems are measured on the same machine. Our consumer GPU is subject to *thermal throttling* (it lowers its clock frequency under sustained load to stay within its temperature limit), so per-sentence latency and per-run energy carry roughly $\pm 15\%$ run-to-run variance. Therefore, we read the per-system averages as indicative and do not over-interpret individual per-dataset cells. The quality metrics, in contrast, are fully deterministic: with the fixed seed and the released model, no stage of the OPENER pipeline is stochastic at inference, so its AMI and F_1 reproduce exactly across re-runs (we verified identical CrossNER scores on an independent re-run), and only latency and energy fluctuate. Because inference is deterministic, the only remaining source of uncertainty in quality scores is the finite test set. We quantify it with a non-parametric bootstrap: for three representative datasets, we resample the gold mentions with replacement ($K=1000$ resamples) and recompute typing-on-gold AMI, yielding the 95% confidence intervals of [Table S1 in Online Resource 1](#). The uncertainty is small (bootstrap std of at most 1.5 AMI, 95% CIs within roughly ± 3 AMI) and the bootstrap means coincide with the released model’s full-test-set scores, indicating that the reported figures are stable rather than fragile seed-specific artefacts. Table 3 collects the complete OPENER configuration used throughout. The embedder is retrained under each of the three seeds (7, 42, 123) while every other component is fixed.

4.5 Main Results

The tables below, starting with Table 4, report the benchmark on all 13 sets, and Figure 3 plots the per-dataset end-to-end AMI of every system. OWNER is run in its native protocol [10]: a single model is trained once on CoNLL-2003 (88 minutes on our GPU) and applied zero-shot to every target set (cells marked ^c), so CoNLL-2003 is in-domain for it (marked ^a). Excluding CoNLL-2003, in-domain for both OWNER and OPENER, OWNER’s 34.3 mean becomes a 32.9 transfer mean, against 39.1 for OPENER-ZS_{trans} on the same 12 sets, a margin that widens from +5.2 to +6.2 AMI. One asymmetry must be kept in mind: OPENER-Sup fits a linear probe on each target’s training split, whereas OWNER, GLiNER and GNER use no target labels. OPENER-Sup thus needs labels but stays cheap at test time, while OWNER needs none but re-encodes and re-clusters every mention at inference, an order of magnitude above OPENER in latency and energy (Table 5 and Figure 4, Table 6 and Figure 5). OPENER-ZS closes this asymmetry. Its un-refined nearest-centroid form, OPENER-ZS_{ind}, already sits within half a point of OWNER end-to-end (33.8 vs 34.3) and matches it on gold (43.2 vs 43.0, subsection 4.7) at a far lower cost (137 ms and 1.85 Wh vs 616 ms and 14.3 Wh). The transductive refinement and the fusion with the detector’s own labels then build OPENER-ZS_{trans}, which reaches 39.5 end-to-end, the best zero-shot system, ahead of GLiNER-L (38.9) and within 0.6 AMI of OPENER-Sup, without any target label. The only generative LLM that fits our 6 GB GPU, Qwen2.5-1.5B-Instruct [31] in 4-bit, trails every dedicated system (26.1 mean AMI on a 150-sentence subsample) while being the slowest per sentence (5.1 s) and the most energy-hungry once normalised by sentence count, about twice GNER’s per-sentence draw (Table 7). OPENER-Sup remains the strongest configuration (40.1 end-to-end, 62.3 on gold), so OPENER-ZS is best read as the frugal, annotation-free operating point of the same pipeline. The four-axis means are plotted in Online Resource 1 (Fig. S1), and a per-domain analysis follows in subsection 4.6.

4.6 Analysis

4.6.1 Where OPENER wins, and why

Two domain patterns stand out across Table 4 and Figure 3.

First, the detector-fusion gains of OPENER-ZS_{trans} concentrate on the context-rich encyclopaedic CrossNER domains (+14.7 AMI on Music, +10.1 on Science, +8.8 on Politics over the un-fused centroid), where the detector’s own zero-shot label is most reliable and lifts OPENER-ZS to detector-level quality (e.g. Music 58.3, edging GLiNER-L’s 58.0).

Second, on specialised schemas with opaque, abbreviation-like labels (FabNER, BioNLP), OPENER’s contrastive typing maps cryptic surface forms far better than prompt-based naming models, as OWNER: OPENER-Sup leads end-to-end (29.8 and 35.9 AMI), and on gold mentions, the gap widens sharply (57.1/69.2 versus OWNER’s 31.9/29.2, subsection 4.7).

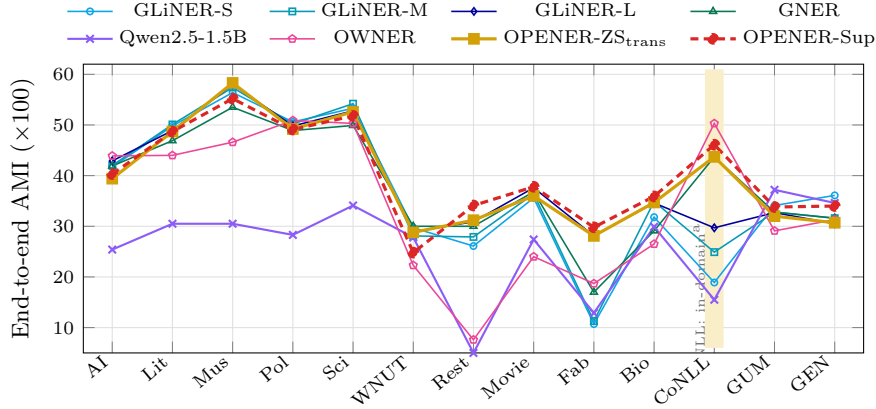
Table 4 Open-world NER, end-to-end. AMI ($\times 100$, $\uparrow$).

Model	AI	Lit	Mus	Pol	Sci	WNUT	Rest	Movie	Fab	Bio	CoNLL	GUM	GEN	Avg
<i>Zero-shot (no target labels)</i>														
GLiNER-S	41.7	49.8	56.4	50.6	53.3	29.7	26.1	35.6	10.7	31.8	18.9	34.1	36.1	36.5
GLiNER-M	42.0	50.1	57.4	50.2	54.2	28.1	27.9	36.4	11.3	34.9	24.9	32.8	31.6	37.1
GLiNER-L	42.9	48.9	58.0	49.8	52.8	28.9	30.9	37.6	28.1	34.6	29.7	32.7	30.5	38.9
GNER	41.9	46.9	53.5	48.9	49.9	30.0	30.0	36.8	17.0	29.1	43.7	32.8	31.6	37.9
Qwen2.5-1.5B ^b	25.4	30.5	30.5	28.3	34.1	27.8	5.0	27.4	12.9	29.9	15.5	37.2	34.6	26.1
OWNER	43.9	44.0	46.6	50.9	50.3	22.3	7.6	24.0	18.7	26.5	50.3 ^a	29.1	31.3	34.3
OPENER-ZS _{ind}	29.7 $\pm$ 2.2	41.6 $\pm$ 2.3	43.6 $\pm$ 0.3	40.4 $\pm$ 0.7	42.5 $\pm$ 0.6	21.9 $\pm$ 1.3	28.0 $\pm$ 1.9	32.8 $\pm$ 0.9	27.6 $\pm$ 0.1	26.2 $\pm$ 1.4	43.5 $\pm$ 1.6	30.8 $\pm$ 1.4	30.1 $\pm$ 1.0	33.8 $\pm$ 0.7
OPENER-ZS _{trans}	39.4 $\pm$ 2.1	48.7 $\pm$ 0.7	58.3 $\pm$ 0.3	49.2 $\pm$ 0.9	52.6 $\pm$ 0.2	28.8 $\pm$ 1.0	31.2 $\pm$ 0.2	36.0 $\pm$ 0.6	28.1 $\pm$ 0.3	34.7 $\pm$ 0.9	43.7 $\pm$ 1.1	32.0 $\pm$ 0.7	30.7 $\pm$ 0.3	39.5 $\pm$ 0.1
<i>Supervised (target labels)</i>														
OPENER-Sup	40.3 $\pm$ 0.2	48.7 $\pm$ 1.3	55.2 $\pm$ 0.8	49.1 $\pm$ 1.4	51.8 $\pm$ 1.7	24.8 $\pm$ 0.2	34.1 $\pm$ 0.9	37.8 $\pm$ 0.2	29.8 $\pm$ 0.5	35.9 $\pm$ 0.5	46.1 $\pm$ 2.1	33.8 $\pm$ 0.1	34.0 $\pm$ 0.3	40.1 $\pm$ 0.5

Abbreviations follow Table 1. Bold marks the best value per column across all systems. Systems are grouped by supervision (*zero-shot* vs the supervised OPENER-Sup). OPENER rows report mean $\pm$ std over the three embedder-retraining seeds (subsection 4.3). The baselines are frozen checkpoints with deterministic inference (single run). Typing-on-gold is reported in Table 9.

^a CoNLL-2003 is in-domain for OWNER (training source) and OPENER (contrastive source) and is not bolded as a transfer result.

^b Qwen2.5-1.5B-Instruct (4-bit) is on a 150-sentence subsample (indicative).

**Fig. 3** End-to-end AMI per dataset (Table 4, OPENER curves show the three-seed means). *Zero-shot* systems are drawn solid. OPENER-Sup is the dashed red line. ^a CoNLL-2003 (shaded) is in-domain for OWNER and OPENER. ^b Qwen is evaluated on a 150-sentence subsample per set**Table 5** Median per-sentence latency (ms, $\downarrow$), end-to-end.

Model	AI	Lit	Mus	Pol	Sci	WNUT	Rest	Movie	Fab	Bio	CoNLL	GUM	GEN	Avg
<i>Zero-shot (no target labels)</i>														
GLiNER-S	21	22	21	21	21	20	19	20	20	22	20	21	21	21
GLiNER-M	36	36	37	38	39	39	38	40	41	42	44	46	45	40
GLiNER-L	144	149	153	149	155	137	132	145	147	143	131	141	137	143
GNER	3431	5408	7461	6472	7110	2906	1610	3344	4310	5034	2914	3435	1500	4226
Qwen2.5-1.5B ^b	4857	6757	9396	7700	8261	3553	2142	5014	2921	4737	2260	4265	3806	5051
OWNER ^c	500	712	895	920	757	311	281	532	438	558	719 ^a	759	626	616
OPENER-ZS _{ind}	109	135	156	172	175	136	127	137	133	136	117	127	124	137
OPENER-ZS _{trans}	218	254	291	277	289	185	173	205	84	98	92	104	97	182
<i>Supervised (target labels)</i>														
OPENER-Sup	222	271	293	159	132	100	92	98	104	108	89	100	94	143

Abbreviations follow Table 1, best per column in bold.

^a CoNLL is OWNER’s in-domain source, measured by a separate inference-only run.

^b Qwen2.5-1.5B-Instruct (4-bit) is on a 150-sentence subsample per set.

^c OWNER exposes no per-sentence call: its figure is an amortised wall-clock mean (its 88-min training is excluded).

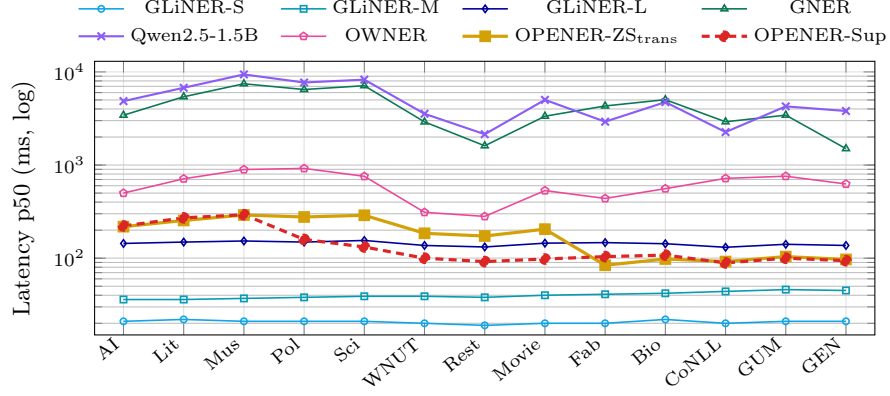


Fig. 4 Per-sentence inference latency (p50, ms, $\log$ scale) per dataset (Table 5). Same systems and styling as Figure 3. OPENER-Sup is the dashed red line. The OPENER variants track GLiNER-L, while GNER and the Qwen LLM are one to two orders of magnitude slower

Table 6 Energy consumption per run (Wh, $\downarrow$), end-to-end.

Model	AI	Lit	Mus	Pol	Sci	WNUT	Rest	Movie	Fab	Bio	CoNLL	GUM	GEN	Avg
<i>Zero-shot (no target labels)</i>														
GLiNER-S	0.19	0.17	0.20	0.27	0.22	0.27	0.35	0.39	0.40	0.42	0.37	0.41	0.40	0.31
GLiNER-M	0.31	0.29	0.33	0.48	0.40	0.50	0.66	0.76	0.77	0.79	0.69	0.82	0.75	0.58
GLiNER-L	0.68	0.67	0.77	1.04	0.91	1.01	1.37	1.51	1.54	1.49	1.38	1.52	1.43	1.18
GNER	17.13	24.28	39.75	50.88	44.76	20.60	17.02	37.17	49.11	63.50	45.22	38.69	26.50	36.51
Qwen2.5-1.5B ^b	12.7	15.1	17.5	18.4	18.0	7.9	4.6	9.5	7.0	11.1	7.3	10.2	9.5	11.4
OWNER ^c	6.6	9.0	12.7	18.3	12.6	6.5	8.6	16.3	13.4	17.1	22.0 ^a	23.2	19.1	14.3
OPENER-ZS _{ind}	0.95	1.06	1.37	1.97	1.69	1.52	2.07	2.28	2.15	2.30	2.10	2.33	2.26	1.85
OPENER-ZS _{trans}	1.14	1.23	1.58	2.08	1.82	1.41	1.82	2.23	1.62	1.87	1.63	1.90	1.79	1.70
<i>Supervised (target labels)</i>														
OPENER-Sup	1.10	1.26	0.87	1.76	1.41	1.31	1.75	1.87	1.96	1.97	1.73	1.91	1.78	1.59

Abbreviations follow Table 1. Best per column in bold.

^a CoNLL is its in-domain training source, measured by a separate inference-only run.

^b Qwen2.5-1.5B-Instruct (4-bit) is on a 150-sentence subsample per set.

^c OWNER is measured at inference only (its one-time 88-min CoNLL-2003 training is amortised, not charged per run).

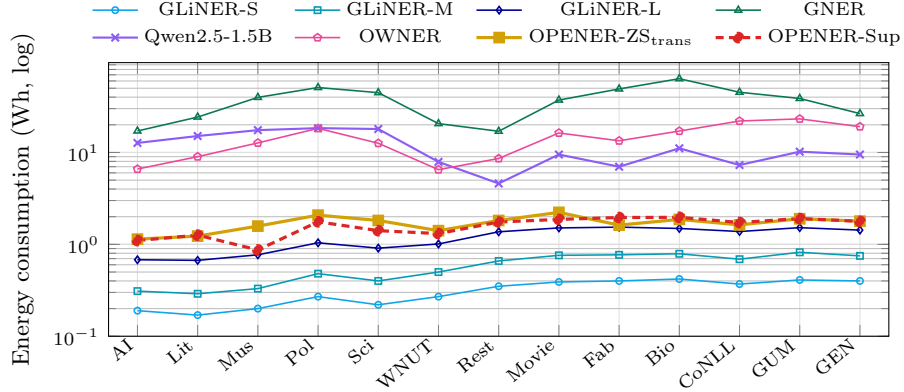


Fig. 5 Per-run energy consumption (Wh, $\log$ scale) per dataset (Table 6). Same systems and styling as Figure 3. The OPENER variants sit an order of magnitude below OWNER and the generative GNER/Qwen baselines

Table 7 Quality vs efficiency, 13-set means.

Model	Params	AMI $\uparrow$	p50 (ms) $\downarrow$	Energy (Wh) $\downarrow$	CO ₂ (g) $\downarrow$
<i>Zero-shot (no target labels)</i>					
GLiNER-S	50M	36.5	21	0.31	0.02
GLiNER-M	200M	37.1	40	0.58	0.03
GLiNER-L	330M	38.9	143	1.18	0.07
GNER	220M	37.9	4226	36.51	2.05
Qwen2.5-1.5B (4-bit) ^b	1.5B	26.1	5051	11.4	0.60
OWNER ^c	294M	34.3	616	14.3	0.74
OPENER-ZS _{ind}	467M	33.8 $\pm$ 0.7	137	1.85	0.10
OPENER-ZS _{trans}	467M	39.5 $\pm$ 0.1	182	1.70	0.10
<i>Supervised (target labels)</i>					
OPENER-Sup	467M	40.1 $\pm$ 0.5	143	1.59	0.09

AMI ($\times 100$, $\uparrow$, OPENER rows: mean $\pm$ std over three retraining seeds). P50 latency (ms), energy (Wh), carbon (gCO₂eq) per run ($\downarrow$, measured on the released seed-42 run). Parameters (end-to-end OPENER = GLiNER-L detector 330M + embedder 137M = 467M, or 137M on gold, OWNER $\approx$ 294M, GLiNER parameter counts from the released checkpoints).

^b Qwen2.5-1.5B-Instruct (4-bit) is on a 150-sentence subsample per set.

^c OWNER’s latency is an amortised wall-clock mean and its one-time 88-min training is not charged per run (Table 5).

4.6.2 Detection is the bottleneck, worst on specialised domains

Contrasting typing on gold mentions with the end-to-end scores isolates the cost of detection. For OPENER-Sup, the mean drop from gold to end-to-end is 22.2 AMI, but it is far from uniform: it averages 18.8 on the encyclopaedic CrossNER domains against 29.9 on the specialised schemas (BioNLP, MIT-Restaurant, MIT-Movie, FabNER), peaking at 33.3 on BioNLP. The typing head is strong everywhere (gold AMI above 50 on 11 of 13 sets). What caps end-to-end quality is the open-world detector’s recall on cryptic, domain-specific surface forms, which confirms detection, not typing, as the dominant bottleneck. The clearest next step for this method is therefore to build and train a stronger mention detector, which we expect to raise the end-to-end scores drastically.

4.6.3 What remains hard

The residual typing errors are dominated by confusions between fine-grained, lexically close types within a domain rather than across domains. The most frequent type confusions are DNA vs. protein (BioNLP), the FabNER process labels CONPRI vs. MATE/PRO/MANP, and the Movie attributes TITLE vs. GENRE/RATING: precisely the opaque schemas of Table 1 where even the surrounding context barely disambiguates the type.

4.6.4 Contrastive learning in action

Figure 2 visualises the effect of the two fine-tuning stages on a *held-out* domain (WNUT-17), never seen during contrastive training. Frozen and unmodified, the

Nomic embedding intermixes entity types. The contrastive step already pulls same-type mentions together, and the hard-negative mining pass sharpens the boundaries between the types it most often confuses, yielding compact, well-separated clusters. The type geometry learned on CoNLL thus transfers to an unseen domain, which is what makes the cheap label-name and linear typing heads effective.

4.7 Ablations

We justify each architectural choice with a one-factor-at-a-time search (Table 8): starting from the default pipeline, we vary one component, hold the others fixed and keep the setting with the highest mean AMI. On the embedder axis, the error-driven hard-negative mining pass, which fine-tunes the contrastive embedder on its most confused type pairs (e.g. *writer/person*, *album/band*), raises typing-on-gold AMI from 54.0 to 60.1, and scaling it (8000 triplets, 3 epochs) reaches 62.5 (+8.5, better on 12 of 13 sets), our selected configuration.

Among frozen base encoders, Nomic v1.5 is the most discriminative (25.8 AMI, ahead of E5, mxbai, MPNet and BGE), but frozen embeddings remain weak and the contrastive step is what makes the space usable (+28.2). The detector trades quality for cost (GLiNER-S to L: 37.7 to 40.2 AMI at 51 to 143 ms), and OPENER’s typing on GLiNER-L’s spans (40.2) even beats GLiNER-L’s own typing (38.9). A closed-set detector (BERT or RoBERTa-NER trained on CoNLL) collapses to near-zero recall outside PER/ORG/LOC/MISC, dragging macro- F_1 to 17.9/24.6 (vs 30.5), which confirms that open-world detection is required. For typing, the balanced linear SVM edges out a per-class GMM and logistic regression (62.5 vs 56.1 vs 55.6).

Replacing it with the label-name nearest centroid removes target supervision at a cost of 18.1 AMI (62.5 to 44.4). This OPENER-ZS_{ind}head stays on par with OWNER on gold mentions (43.2 vs 43.0 over three seeds, Table 9) while fitting in 293 ms instead of 2.2 s. The transductive refinement recovers 7.0 AMI on gold (44.4 to 51.4) and 2.6 end-to-end at the released seed (34.2 to 36.8), a transductive rather than inductive setting since it exploits the unlabelled test distribution. Aligning mentions and label names on the $\approx 4.6k$ types of Pile-NER [50] does not help (42.2 vs 44.4, Table S2 in Online Resource 1), and a label-name hard-mining pass degrades it further (39.7), because Pile-NER’s GPT-generated types contain near-synonyms (e.g. *person/per*) that the mining pulls apart. We therefore keep the target-tuned embedder for OPENER-ZS.

Training cost forms a separate efficiency axis (Fit column): inference is governed by the detector alone, whereas fitting the typing head is $7\times$ cheaper at dimension 64 than at 768 (for 4.6 AMI less) and $4.5\times$ cheaper with balanced logistic regression than with the linear SVM. On gold mentions (Table 9), one cell stands out: OPENER-ZS_{ind} drops to 16.1 AMI on MIT-Movie (± 7.1), as attribute-like label names (PLOT, REVIEW, TRAILER) carry little lexical signal when every mention is movie-related. The transductive refinement recovers most of this gap (44.8), and the higher end-to-end value (32.8) is not directly comparable, since the end-to-end sentinels reshape both partitions.

Figure 6 shows that quality degrades smoothly as d shrinks: $d=256$ already reaches 61.2 AMI (within 1.3 of the full 62.5). Truncation thus trades a little quality for a large

Table 8 OPENER architecture search: one component is varied at a time around the default pipeline (**bold**).

Configuration	AMI $\uparrow$	F ₁ $\uparrow$	Fit (ms) $\downarrow$	p50 (ms) $\downarrow$	En. (Wh) $\downarrow$	CO ₂ (g) $\downarrow$
<i>Mention detector</i> (each row: that detector $\rightarrow$ OPENER embedder + typing head)						
GLiNER-S	37.7	28.8	0.0	51	0.73	0.04
GLiNER-M	38.0	28.7	0.0	70	1.01	0.06
GLiNER-L (open-world)	40.2	30.5	0.0	143	1.59	0.09
BERT-NER (closed-set) [7]	33.9	17.9	0.0	68	0.58	0.03
RoBERTa-NER (closed-set) [23]	39.4	24.6	0.0	54	0.58	0.04
<i>Entity embedder</i> ^a						
frozen bge-base-en-v1.5 [46]	23.6	41.9	0	43	0.63	0.04
frozen all-mpnet-base-v2 [37]	24.1	42.6	0	52	0.85	0.05
frozen mxbai-embed-large-v1 [20]	24.4	42.5	0	363	3.86	0.25
frozen e5-base-v2 [43]	25.3	43.4	0	39	0.59	0.04
frozen Nomic v1.5 (selected base)	25.8	43.9	0	42	0.63	0.04
+ contrastive (Triplet, CoNLL, 3 ep.)	54.0	63.3	0	42	0.63	0.04
+ hard-negative mining (2 ep., 3k tr.)	60.1	68.8	0	42	0.63	0.04
+ hard-negative mining (3 ep., 8k tr.)	62.5	71.8	0	42	0.63	0.04
<i>Matryoshka dimension</i> ^b						
64	57.9	67.7	299	42	0.63	0.04
128	59.9	69.7	562	42	0.63	0.04
256	61.2	70.6	926	42	0.63	0.04
512	62.0	71.2	1556	42	0.63	0.04
768	62.5	71.8	2176	42	0.63	0.04
<i>Entity typing</i> ^c						
GMM (per-class)	56.1	65.3	527	42	0.63	0.04
LogReg	55.6	53.5	546	42	0.63	0.04
LogReg (balanced)	57.8	67.5	485	42	0.63	0.04
LinearSVC	61.7	66.5	2041	42	0.63	0.04
LinearSVC (balanced)	62.5	71.8	2217	42	0.63	0.04
Nearest centroid (zero-shot) ^d	44.4	46.8	293	42	0.63	0.04
+ transductive refine ^d	51.4	48.5	293	42	0.63	0.04

Metrics. AMI and macro- F_1 ($\times 100$, $\uparrow$). Fit (ms) is the typing-head training time. Latency, energy and carbon are per row ($\downarrow$). All rows use the released seed (42).

Protocol, which differs per axis. On the *mention-detector* axis each row is OPENER's *end-to-end* AMI with that detector, which only proposes spans that the OPENER head re-types (unlike the standalone systems of Table 4). The ^a embedder, ^c typing and ^b dimension axes are instead *typing-on-gold* over the 13 sets.

Cost. Latency and energy are the deployed end-to-end cost on the mention-detector axis, and the constant embedder-plus-typing cost on the other axes.

^b Truncates the Matryoshka vector after encoding.

^d Zero-shot nearest-centroid head (no target labels). Its Fit is the one-off prototype-embedding cost.

Table 9 Entity typing on gold mentions (detection bypassed): OPENER vs OWNER (AMI $\times 100$, $\uparrow$).

Model	AI	Lit	Mus	Pol	Sci	WNUT	Rest	Movie	Fab	Bio	CoNLL	GUM	GEN	Avg
<i>Zero-shot (no target labels)</i>														
OWNER	55.7	54.3	59.8	63.2	63.1	30.5	29.3	31.8	31.9	29.2	52.5 ^a	30.0	27.2	43.0
OPENER-ZS _{ind}	38.7 $\pm$ 3.6	57.7 $\pm$ 3.3	54.7 $\pm$ 1.7	57.4 $\pm$ 5.8	57.8 $\pm$ 1.6	31.3 $\pm$ 2.6	38.1 $\pm$ 4.6	16.1 $\pm$ 7.1	30.4 $\pm$ 5.1	51.1 $\pm$ 2.9	63.9 $\pm$ 3.0	33.7 $\pm$ 3.8	31.0 $\pm$ 2.5	43.2 $\pm$ 1.3
OPENER-ZS _{trans}	46.1 $\pm$ 2.0	63.5 $\pm$ 2.3	67.0 $\pm$ 1.9	66.4 $\pm$ 1.2	59.6 $\pm$ 2.0	32.7 $\pm$ 2.8	47.3 $\pm$ 4.8	44.8 $\pm$ 1.1	33.9 $\pm$ 3.3	49.8 $\pm$ 0.3	70.7 $\pm$ 1.7	41.5 $\pm$ 2.5	38.8 $\pm$ 1.6	50.9 $\pm$ 0.4
<i>Supervised (target labels)</i>														
OPENER-Sup	57.6$\pm$0.9	66.0$\pm$2.0	72.3$\pm$1.5	73.6$\pm$1.8	69.4$\pm$2.2	38.8$\pm$1.0	63.2$\pm$2.0	67.7$\pm$1.9	57.1$\pm$2.0	69.2$\pm$1.8	73.3$\pm$1.6	49.8$\pm$0.6	51.5$\pm$0.2	62.3$\pm$1.0

Reading. Abbreviations follow Table 1. Bold marks the best value per column. Systems are grouped by supervision.

Statistics. OPENER rows report mean $\pm$ std over the three embedder-retraining seeds (subsection 4.3). OWNER is a single training run.

Typing heads. OPENER-ZS_{ind} is the un-refined nearest centroid and OPENER-ZS_{trans} adds the transductive refinement. Note that the detector fusion of Table 4 is end-to-end-only, so it is inactive here, where detection is bypassed. OPENER-Sup is the supervised upper bound.

Outlier. The low OPENER-ZS_{ind} MIT-Movie cell (16.1 $\pm$ 7.1) is analysed in subsection 4.7: the schema's attribute-like label names give the prototypes little lexical signal, and the sentinel-based end-to-end score of Table 4 is not directly comparable to this gold score.

^a CoNLL-2003 is in-domain for OWNER (training source) and OPENER (contrastive source) and is not bolded.

cut in the typing-head fit cost, and the Matryoshka structure survives our contrastive fine-tuning.

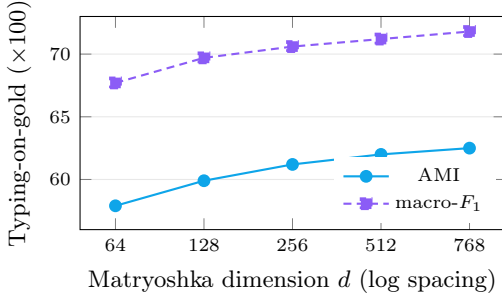


Fig. 6 Typing-on-gold quality vs the Matryoshka truncation dimension d . Both AMI and macro- F_1 degrade smoothly, with diminishing returns rather than a cliff, and flatten above $d=256$. Truncation is therefore a graceful cost/quality knob, and the Matryoshka structure survives the contrastive fine-tuning. Data from Table 8 (dimension axis, released seed, 13-set means, log-spaced d)

5 Conclusion

We presented OPENER, an open-world NER pipeline assembled entirely from ready-to-use components (a frozen detector, a pretrained Matryoshka embedder sharpened by a light contrastive fine-tuning with error-driven hard-negative mining, and a swappable typing head), with no encoder trained from scratch. Its modularity let us select every component on measured evidence, and the Matryoshka structure survives the fine-tuning as a compute/quality knob (Figure 6).

Both typing heads perform strongly, at opposite ends of the supervision axis. OPENER-ZS needs no target labels and is the best of the compared zero-shot systems end-to-end (39.5 AMI), at one to two orders of magnitude lower energy than the trained-encoder and generative baselines. OPENER-Sup, a balanced linear probe for fixed label sets, is the most accurate system both on gold mentions (62.3 vs 43.0 AMI for OWNER) and end-to-end (40.1 AMI). These results rest on a 13-dataset benchmark that reports quality, latency and energy jointly and keeps typing-on-gold separate from end-to-end.

Two findings carry beyond this system. First, an open-world typing head need not train its own encoder: a general-purpose embedding sharpened by a light contrastive step is discriminative enough to type entities. Second, the bottleneck lies in detection rather than typing: typing reaches 62.3 AMI on gold mentions but 40.1 end-to-end as detector recall falls on specialised domains, which makes the detector the clearest axis for future gains.

The approach has clear limits. Supervised typing needs target labels, and OPENER-ZS depends on the quality of the label names and, in its transductive form, on access to the unlabelled test mentions. Our experiments run on a single consumer GPU with 6 GB of VRAM, which bounded the budget (test splits capped at 1000 sentences). The quality scores are nonetheless stable across three retraining seeds (std of 0.1 to 1.3 AMI) and across test-set bootstrap resamples, whereas latency and energy vary by about $\pm 15\%$ and are read as averages. Future work includes stronger mention detection on specialised domains and extending the detect-then-type design to entity linking and relation extraction.

Statements and Declarations

Funding. No funding was received for conducting this study.

Competing interests. The authors have no relevant financial or non-financial interests to disclose.

Ethics approval, consent to participate and consent for publication. Not applicable.

Data and code availability. All datasets are public benchmarks cited in subsection 4.1. The code, configurations and aggregated results are available at https://github.com/Thibault-GAREL/LyRIDS_OPENER, and the fine-tuned embedders at <https://huggingface.co/Thibault-GAREL/opener-zs> and <https://huggingface.co/Thibault-GAREL/opener-sup>.

Author contributions. T.G. conceived the study, designed the methodology, developed the software, carried out the experiments, curated the data, performed the formal analysis and validation, provided the computing resources, prepared the figures and tables, and wrote and revised the manuscript. F.B. and A.S. jointly supervised the work. All authors reviewed the manuscript and approved the final version.

References

- [1] Aly R, Vlachos A, McDonald R (2021) Leveraging type descriptions for zero-shot named entity recognition and classification. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP), pp 1516–1528, <https://doi.org/10.18653/v1/2021.acl-long.120>
- [2] Aoyama T, Behzad S, Gessler L, et al (2023) GENTLE: A genre-diverse multilayer challenge set for english NLP and linguistic evaluation. In: Proceedings of the 17th Linguistic Annotation Workshop (LAW-XVII), pp 166–178, <https://doi.org/10.18653/v1/2023.law-1.17>
- [3] Das SSS, Katiyar A, Passonneau RJ, et al (2022) CONTaiNER: Few-shot named entity recognition via contrastive learning. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL), pp 6338–6353, <https://doi.org/10.18653/v1/2022.acl-long.439>
- [4] Dempster AP, Laird NM, Rubin DB (1977) Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society: Series B (Methodological)* 39(1):1–22. <https://doi.org/10.1111/j.2517-6161.1977.tb01600.x>
- [5] Derczynski L, Nichols E, van Erp M, et al (2017) Results of the WNUT2017 shared task on novel and emerging entity recognition. In: Proceedings of the 3rd Workshop on Noisy User-generated Text (W-NUT), pp 140–147, <https://doi.org/10.18653/v1/w17-4418>
- [6] Dettmers T, Pagnoni A, Holtzman A, et al (2023) QLoRA: Efficient finetuning of quantized LLMs. In: *Advances in Neural Information Processing Systems 36* (NeurIPS). Curran Associates, Inc., <https://doi.org/10.52202/075280-0441>
- [7] Devlin J, Chang MW, Lee K, et al (2019) BERT: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), pp 4171–4186, <https://doi.org/10.18653/v1/n19-1423>

- [8] Ding Y, Li J, Wang P, et al (2024) Rethinking negative instances for generative named entity recognition. In: Findings of the Association for Computational Linguistics: ACL 2024, pp 3461–3475, <https://doi.org/10.18653/v1/2024.findings-acl.206>
- [9] Fan RE, Chang KW, Hsieh CJ, et al (2008) LIBLINEAR: A library for large linear classification. *Journal of Machine Learning Research* 9:1871–1874
- [10] Genest PY, Portier PE, Egyed-Zsigmond E, et al (2025) OWNER: Toward unsupervised open-world named entity recognition. *IEEE Access* 13:50077–50105. <https://doi.org/10.1109/ACCESS.2025.3552122>
- [11] He P, Gao J, Chen W (2023) DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. In: The Eleventh International Conference on Learning Representations (ICLR)
- [12] Henderson P, Hu J, Romoff J, et al (2020) Towards the systematic reporting of the energy and carbon footprints of machine learning. *Journal of Machine Learning Research* 21:1–44
- [13] Kim JD, Ohta T, Tateisi Y, et al (2003) GENIA corpus—a semantically annotated corpus for bio-textmining. *Bioinformatics* 19(Suppl. 1):i180–i182. <https://doi.org/10.1093/bioinformatics/btg1023>
- [14] Kim JD, Ohta T, Tsuruoka Y, et al (2004) Introduction to the bio-entity recognition task at JNLPBA. In: Proceedings of the International Joint Workshop on Natural Language Processing in Biomedicine and its Applications (NLPBA/BioNLP), pp 70–75, <https://doi.org/10.3115/1567594.1567610>
- [15] Kumar A, Starly B (2022) FabNER: Information extraction from manufacturing process science domain literature using named entity recognition. *Journal of Intelligent Manufacturing* 33(8):2393–2407. <https://doi.org/10.1007/s10845-021-01807-x>
- [16] Kusupati A, Bhatt G, Rege A, et al (2022) Matryoshka representation learning. In: Advances in Neural Information Processing Systems 35 (NeurIPS 2022), <https://doi.org/10.52202/068431-2192>
- [17] Lai P, Zhang Z, Zhang W, et al (2025) Enhancing unsupervised sentence embeddings via knowledge-driven data augmentation and Gaussian-decayed contrastive learning. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL), Volume 1: Long Papers. Association for Computational Linguistics, pp 4919–4940, <https://doi.org/10.18653/v1/2025.acl-long.244>
- [18] Lample G, Ballesteros M, Subramanian S, et al (2016) Neural architectures for named entity recognition. In: Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), pp 260–270, <https://doi.org/10.18653/v1/n16-1030>
- [19] Li J, Sun A, Han J, et al (2022) A survey on deep learning for named entity recognition. *IEEE Transactions on Knowledge and Data Engineering* 34(1):50–70. <https://doi.org/10.1109/tkde.2020.2981314>

- [20] Li X, Li J (2024) AoE: Angle-optimized embeddings for semantic textual similarity. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL), pp 1825–1839, <https://doi.org/10.18653/v1/2024.acl-long.101>
- [21] Li X, Feng J, Meng Y, et al (2020) A unified MRC framework for named entity recognition. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), pp 5849–5859, <https://doi.org/10.18653/v1/2020.acl-main.519>
- [22] Liu J, Pasupat P, Cyphers S, et al (2013) Asgard: A portable architecture for multilingual dialogue systems. In: Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp 8386–8390, <https://doi.org/10.1109/icassp.2013.6639301>
- [23] Liu Y, Ott M, Goyal N, et al (2019) RoBERTa: A robustly optimized BERT pretraining approach. arXiv preprint arXiv:1907.11692
- [24] Liu Z, Xu Y, Yu T, et al (2021) CrossNER: Evaluating cross-domain named entity recognition. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp 13452–13460, <https://doi.org/10.1609/aaai.v35i15.17587>
- [25] Luccioni AS, Jernite Y, Strubell E (2024) Power hungry processing: Watts driving the cost of AI deployment? In: Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT), pp 85–99, <https://doi.org/10.1145/3630106.3658542>
- [26] Luccioni S, Schmidt V, Lacoste A, et al (2019) Quantifying the carbon emissions of machine learning. In: NeurIPS 2019 Workshop on Tackling Climate Change with Machine Learning, URL <https://www.climatechange.ai/papers/neurips2019/22>
- [27] Nguyen NTH, Miwa M, Ananiadou S (2023) Span-based named entity recognition by generating and compressing information. In: Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL), pp 1984–1996, <https://doi.org/10.18653/v1/2023.eacl-main.146>
- [28] Nussbaum Z, Morris JX, Duderstadt B, et al (2025) Nomic embed: Training a reproducible long context text embedder. Transactions on Machine Learning Research (TMLR) Published in TMLR 02/2025. OpenReview: <https://openreview.net/forum?id=IPmzyQSiQE>
- [29] Paszke A, Gross S, Massa F, et al (2019) PyTorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems 32 (NeurIPS)
- [30] Pedregosa F, Varoquaux G, Gramfort A, et al (2011) Scikit-learn: Machine learning in Python. Journal of Machine Learning Research 12:2825–2830
- [31] Qwen Team (2025) Qwen2.5 technical report. arXiv preprint arXiv:2412.15115
- [32] Raffel C, Shazeer N, Roberts A, et al (2020) Exploring the limits of transfer learning with a unified text-to-text transformer. Journal of Machine Learning Research 21:1–67

- [33] Reimers N, Gurevych I (2019) Sentence-BERT: Sentence embeddings using siamese BERT-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp 3982–3992, <https://doi.org/10.18653/v1/d19-1410>
- [34] Sainz O, García-Ferrero I, Agerri R, et al (2024) GoLLIE: Annotation guidelines improve zero-shot information-extraction. In: The Twelfth International Conference on Learning Representations (ICLR)
- [35] Schroff F, Kalenichenko D, Philbin J (2015) FaceNet: A unified embedding for face recognition and clustering. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp 815–823, <https://doi.org/10.1109/cvpr.2015.7298682>
- [36] Schwartz R, Dodge J, Smith NA, et al (2020) Green AI. Communications of the ACM 63(12):54–63. <https://doi.org/10.1145/3381831>
- [37] Song K, Tan X, Qin T, et al (2020) MPNet: Masked and permuted pre-training for language understanding. In: Advances in Neural Information Processing Systems (NeurIPS)
- [38] Strubell E, Ganesh A, McCallum A (2019) Energy and policy considerations for deep learning in NLP. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL), pp 3645–3650, <https://doi.org/10.18653/v1/p19-1355>
- [39] Tjong Kim Sang EF, De Meulder F (2003) Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition. In: Proceedings of the Seventh Conference on Natural Language Learning (CoNLL), pp 142–147, <https://doi.org/10.3115/1119176.1119195>
- [40] Touvron H, Lavril T, Izacard G, et al (2023) LLaMA: Open and efficient foundation language models. arXiv preprint arXiv:230213971
- [41] Vinh NX, Epps J, Bailey J (2010) Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance. Journal of Machine Learning Research 11:2837–2854
- [42] Wang G, Li C, Wang W, et al (2018) Joint embedding of words and labels for text classification. In: Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL), pp 2321–2331, <https://doi.org/10.18653/v1/p18-1216>
- [43] Wang L, Yang N, Huang X, et al (2022) Text embeddings by weakly-supervised contrastive pre-training. arXiv preprint arXiv:221203533
- [44] Wei X, Cui X, Cheng N, et al (2023) ChatIE: Zero-shot information extraction via chatting with ChatGPT. arXiv preprint arXiv:230210205
- [45] Wolf T, Debut L, Sanh V, et al (2020) Transformers: State-of-the-art natural language processing. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP), pp 38–45, <https://doi.org/10.18653/v1/2020.emnlp-demos.6>
- [46] Xiao S, Liu Z, Zhang P, et al (2024) C-Pack: Packed resources for general chinese embeddings. In: Proceedings of the 47th International ACM SIGIR Conference

- on Research and Development in Information Retrieval (SIGIR), pp 641–649, <https://doi.org/10.1145/3626772.3657878>
- [47] Yu J, Bohnet B, Poesio M (2020) Named entity recognition as dependency parsing. In: Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), pp 6470–6476, <https://doi.org/10.18653/v1/2020.acl-main.577>
 - [48] Zaratiana U, Tomeh N, Holat P, et al (2024) GLiNER: Generalist model for named entity recognition using bidirectional transformer. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Volume 1: Long Papers. Association for Computational Linguistics, pp 5364–5376, <https://doi.org/10.18653/v1/2024.naacl-long.300>
 - [49] Zeldes A (2017) The GUM corpus: Creating multilayer resources in the classroom. *Language Resources and Evaluation* 51(3):581–612. <https://doi.org/10.1007/s10579-016-9343-x>
 - [50] Zhou W, Zhang S, Gu Y, et al (2024) UniversalNER: Targeted distillation from large language models for open named entity recognition. In: The Twelfth International Conference on Learning Representations (ICLR)